

Solana Esports Platform

Monetized Matchmaking, MPC Wallets & Prize Distribution

Production-Grade Implementation

Agenda

1. **System Overview** - Architecture & Components
2. **Service Layer Architecture** - Core Services Deep Dive
3. **MPC Wallet Integration** - Secure Player Wallets
4. **Matchmaking System** - Entry Fees & Escrow
5. **Prize Distribution** - Calculations & Strategies
6. **Tournament Management** - Bracket Generation
7. **Security Practices** - Production Considerations
8. **Implementation Details** - Code Walkthrough

System Architecture Overview



Service Layer Architecture

Five Core Services



PlayerWalletService

MPC-Secured Wallet Management

```
@Injectable()
export class PlayerWalletService {
  // Creates MPC wallet with 2-of-3 threshold scheme
  async createWallet(request: CreatePlayerWalletRequest) {
    const mpcWallet = await this.mpcService.createMpcWallet({
      name: `Player Wallet - ${playerId}`,
      thresholdScheme: ThresholdScheme.TSS_2_3,
      participants: [
        { participantId: `player_${playerId}`, ... }, // Device key
        { participantId: `platform_signer`, ... },    // Platform HSM
        { participantId: `recovery_service`, ... },   // Cold storage
      ],
    });
    // Store wallet with default limits
    return this.walletRepository.create({
      playerId,
      mpcWalletId: mpcWallet.walletId,
      dailyWithdrawalLimit: DEFAULT_DAILY_LIMIT, // 10 SOL
      status: PlayerWalletStatus.ACTIVE,
    });
  }
}
```



PlayerWalletService Methods

Method	Purpose	Key Logic
<code>createWallet</code>	Initialize MPC wallet	2-of-3 threshold scheme
<code>deposit</code>	Record external deposit	Verify on-chain signature
<code>withdraw</code>	MPC-signed withdrawal	Rate limit + daily cap
<code>lockFunds</code>	Reserve for escrow	Move to <code>lockedBalance</code>
<code>unlockFunds</code>	Release from escrow	Restore to <code>availableBalance</code>
<code>creditPrize</code>	Award winnings	Direct credit + audit log

Balance Tracking

```
interface WalletBalance {
  availableBalance: string; // Spendable
  lockedBalance: string;    // In active matches
  totalDeposited: string;   // Lifetime in
  totalWithdrawn: string;   // Lifetime out
  totalWinnings: string;    // Prize earnings
}
```

EscrowService

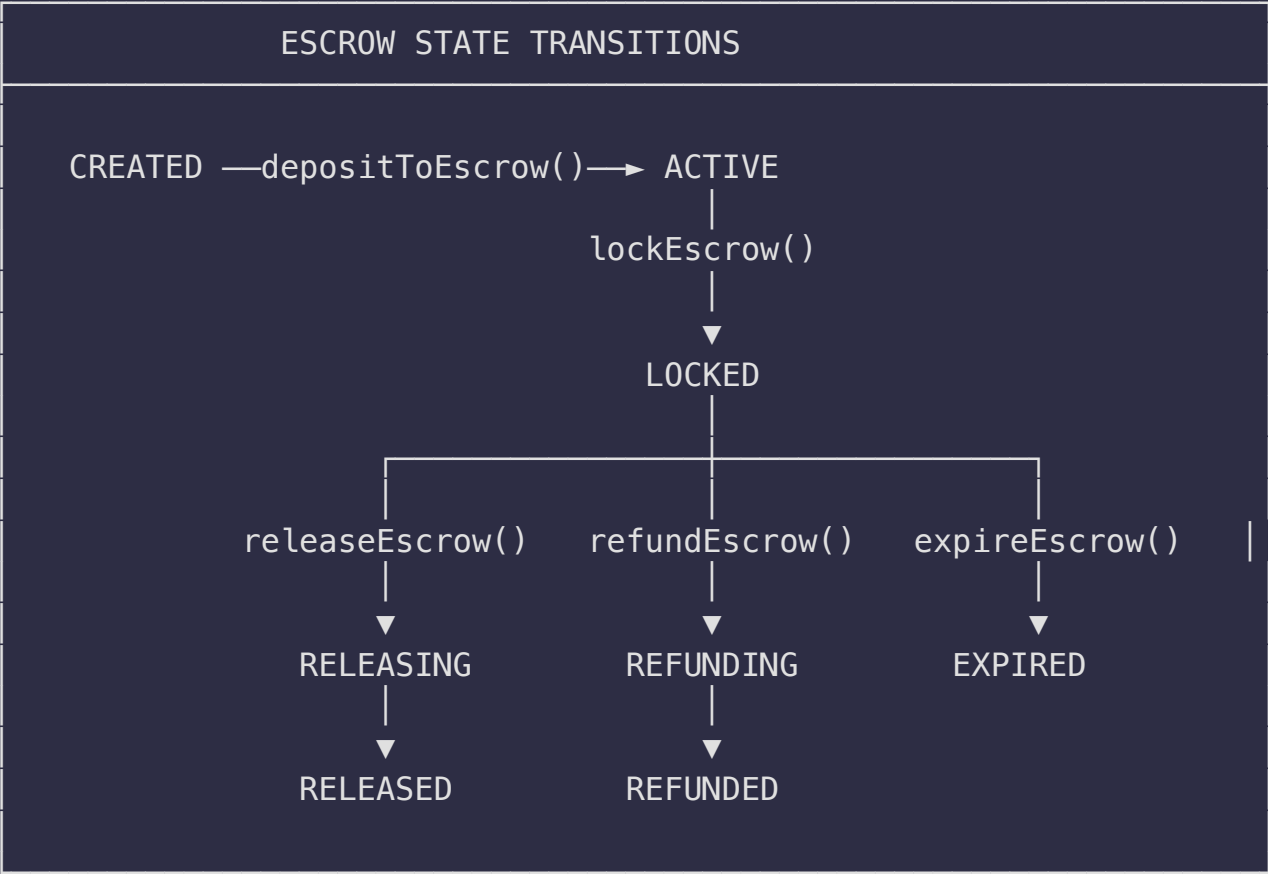
Trustless Fund Management

```
@Injectable()
export class EscrowService {
  // Generate deterministic escrow address (PDA simulation)
  private generateEscrowAddress(sourceType, sourceId): string {
    const seed = `${sourceType}:${sourceId}`;
    return createHash('sha256')
      .update(seed)
      .digest('hex')
      .slice(0, 44);
  }

  async createEscrow(request: CreateEscrowRequest) {
    const escrowAddress = this.generateEscrowAddress(
      request.sourceType,
      request.sourceId
    );
    return this.escrowRepository.create({
      escrowId: `escrow_${randomBytes(8).toString('hex')}`,
      escrowAddress,
      sourceType,           // MATCH or TOURNAMENT
      sourceId,             // match_xxx or tournament_xxx
      platformFeePercent: 5.0,
      status: EscrowStatus.CREATED,
    });
  }
}
```



EscrowService State Machine



MatchmakingService

Core Match Operations

```
@Injectable()
export class MatchmakingService {
  async createMatch(request: CreateMatchRequest): Promise<Match> {
    // 1. Validate entry fee bounds
    const minFee = BigInt(process.env.MIN_ENTRY_FEE || '1000000');
    const maxFee = BigInt(process.env.MAX_ENTRY_FEE || '100000000000');

    // 2. Create escrow account
    const escrow = await this.escrowService.createEscrow({
      sourceType: EscrowSourceType.MATCH,
      sourceId: matchId,
      platformFeePercent,
    });

    // 3. Create match entity
    return this.matchRepository.create({
      matchId,
      gameType,
      entryFee,
      status: MatchStatus.CREATED,
      svmAddress: escrow.escrowAddress,
    });
  }
}
```



MatchmakingService - Join Flow

```
async joinMatch(request: JoinMatchRequest): Promise<MatchParticipant> {
  const match = await this.getMatchById(matchId);

  // 1. Validate match is joinable
  if (!match.isJoinable()) {
    throw new BadRequestException('Match not accepting players');
  }

  // 2. Get player wallet and verify balance
  const wallet = await this.playerWalletService.getWallet(playerId);
  if (BigInt(wallet.availableBalance) < BigInt(match.entryFee)) {
    throw new BadRequestException('Insufficient balance');
  }

  // 3. Lock entry fee in player wallet
  await this.playerWalletService.lockFunds({
    playerId,
    amount: match.entryFee,
    reference: matchId,
  });

  // 4. Deposit to escrow
  await this.escrowService.depositToEscrow({
    escrowId,
    walletId: wallet.id,
    amount: match.entryFee,
  });

  // 5. Update prize pool
  match.prizePool = (BigInt(match.prizePool) + BigInt(entryFee)).toString();
}
```

PrizeDistributionService

Prize Calculation Strategy

```
@Injectable()
export class PrizeDistributionService {
  async distributeMatchPrizes(match, participants): Promise<PrizeDistribution> {
    // 1. Get escrow balance
    const escrowBalance = await this.escrowService.getEscrowBalance(escrowId);
    const totalPrizePool = BigInt(escrowBalance.currentBalance);

    // 2. Calculate platform fee (basis points for precision)
    const platformFee = (totalPrizePool *
      BigInt(Math.round(match.platformFeePercent * 100))) /
      BigInt(10000);

    // 3. Calculate distributable amount
    const distributableAmount = totalPrizePool - platformFee;

    // 4. Calculate individual distributions
    const distributions = this.calculateMatchDistributions(
      match, participants, distributableAmount
    );

    // 5. Release escrow and credit wallets
    await this.escrowService.releaseEscrow({ escrowId, distributions });

    for (const dist of distributions) {
      await this.playerWalletService.creditPrize(
        dist.playerId, dist.amount, match.matchId
      );
    }
  }
}
```

Prize Calculation Methods

Match Distribution Strategies

```
private calculateMatchDistributions(  
  match: Match,  
  participants: MatchParticipant[],  
  distributableAmount: bigint  
): PrizeCalculation[] {  
  const distributions: PrizeCalculation[] = [];  
  
  // Strategy 1: Duel (1v1) – Winner Takes All  
  if (match.gameType === 'duel' && match.result?.winnerIds?.length === 1) {  
    const winner = participants.find(p => p.playerId === winnerId);  
    distributions.push({  
      walletId: winner.walletId,  
      playerId: winner.playerId,  
      placement: 1,  
      amount: distributableAmount.toString(),  
      percentage: 100,  
    });  
  }  
  
  // Strategy 2: Team/Multi-Winner – Equal Split  
  else if (match.result?.winnerIds?.length > 1) {  
    const winnerCount = match.result.winnerIds.length;  
    const prizePerWinner = distributableAmount / BigInt(winnerCount);  
    // ... distribute equally  
  }  
}
```



Tournament Prize Structure

Configurable Payout Tiers

```
private calculateTournamentDistributions(  
  tournament: Tournament,  
  registrations: TournamentRegistration[],  
  distributableAmount: bigint  
): PrizeCalculation[] {  
  const distributions: PrizeCalculation[] = [];  
  
  // Sort by final placement  
  const ranked = registrations  
    .filter(r => r.finalPlacement != null)  
    .sort((a, b) => a.finalPlacement - b.finalPlacement);  
  
  // Apply prize structure  
  for (const prizeEntry of tournament.prizeStructure) {  
    const registration = ranked.find(  
      r => r.finalPlacement === prizeEntry.place  
    );  
  
    if (registration) {  
      // Support both percentage and fixed amounts  
      const amount = prizeEntry.fixedAmount  
        ? BigInt(prizeEntry.fixedAmount)  
        : (distributableAmount * BigInt(prizeEntry.percentage)) / 100n;  
  
      distributions.push({  
        playerId: registration.playerId,  
        placement: prizeEntry.place,  
        amount: amount.toString(),  
        percentage: prizeEntry.percentage,  
      });  
    }  
  }  
}
```



Fee Calculation Deep Dive

Platform Fee with Basis Points

```
// Why basis points? Precision without floating point errors

// BAD: Floating point multiplication
const fee = totalPool * 0.05; // May have rounding errors

// GOOD: Basis points (100 basis points = 1%)
const platformFee = (totalPrizePool *
  BigInt(Math.round(platformFeePercent * 100))) /
  BigInt(10000);

// Example: 5% fee on 2 SOL (2,000,000,000 lamports)
// platformFeePercent = 5.0
// Math.round(5.0 * 100) = 500 (basis points)
// (2,000,000,000 * 500) / 10000 = 100,000,000 lamports
// = 0.1 SOL platform fee
// Winner receives: 1.9 SOL
```

Pool Size	Fee %	Platform Fee	Winner Prize
2 SOL	5%	0.1 SOL	1.9 SOL

TournamentService

Bracket Generation Algorithm

```
@Injectable()
export class TournamentService {
  async generateBracket(tournamentId: string): Promise<Tournament> {
    const tournament = await this.getTournamentById(tournamentId);
    const registrations = await this.registrationRepository.find({
      where: { tournamentId: tournament.id },
    });

    // Seed players (by skill rating or random)
    const seededPlayers = this.seedPlayers(registrations);

    // Generate bracket based on type
    let bracket: BracketNode[];
    switch (tournament.bracketType) {
      case BracketType.SINGLE_ELIMINATION:
        bracket = this.generateSingleElimination(seededPlayers);
        break;
      case BracketType.DOUBLE_ELIMINATION:
        bracket = this.generateDoubleElimination(seededPlayers);
        break;
      case BracketType.ROUND_ROBIN:
        bracket = this.generateRoundRobin(seededPlayers);
        break;
    }

    // Create match entities for each bracket slot
    await this.createBracketMatches(tournament, bracket);
  }
}
```



Single Elimination Bracket

Seeded Matchups (Standard 1v8, 2v7, etc.)

```
private generateSingleElimination(  
  players: TournamentRegistration[]  
) : BracketMatch[] {  
  const n = players.length;  
  const rounds = Math.ceil(Math.log2(n));  
  const bracket: BracketMatch[] = [];  
  
  // First round matchups (seeded)  
  // Seed 1 vs Seed 8, Seed 2 vs Seed 7, etc.  
  for (let i = 0; i < n / 2; i++) {  
    const highSeed = players[i];  
    const lowSeed = players[n - 1 - i];  
  
    bracket.push({  
      round: 1,  
      position: i,  
      player1Id: highSeed?.playerId,  
      player2Id: lowSeed?.playerId,  
      nextMatchPosition: Math.floor(i / 2),  
    });  
  }  
  
  // Generate placeholder matches for subsequent rounds  
  for (let round = 2; round <= rounds; round++) {  
    const matchesInRound = Math.pow(2, rounds - round);  
    for (let i = 0; i < matchesInRound; i++) {  
      bracket.push({ round, position: i, player1Id: null, player2Id: null });  
    }  
  }  
}
```


MPC Wallet Architecture

Multi-Party Computation (2-of-3 Threshold)





MPC Benefits for Gaming

Feature	Traditional Wallet	MPC Wallet
Single Point of Failure	✗ Yes	✓ No
Key Recovery	✗ Complex	✓ Built-in
Account Takeover Risk	✗ High	✓ Mitigated
User Experience	✗ Seed Phrases	✓ Seamless
Platform Control	✗ None	✓ Fraud Prevention

Key Advantages:

- **No seed phrase exposure** to players
- **Platform co-signing** prevents fraud
- **Recovery service** for lost devices
- **Rate limiting** on withdrawals



Player Wallet Entity

```
@Entity('player_wallets')
export class PlayerWallet {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  playerId: string;

  @Column()
  mpcWalletId: string; // Link to MPC system

  @Column()
  publicKey: string; // Solana address

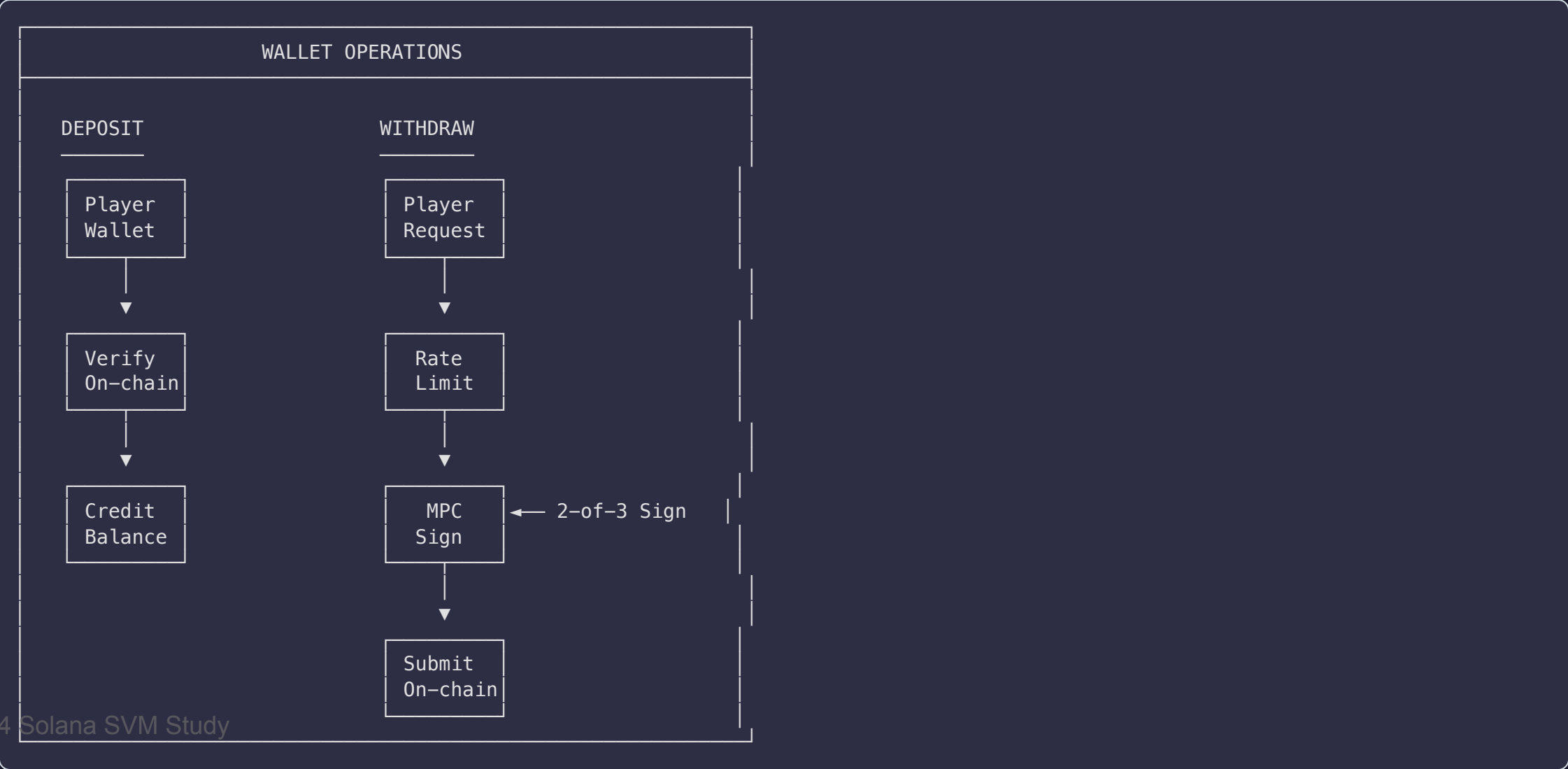
  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  availableBalance: string; // Lamports

  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  lockedBalance: string; // In escrow

  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  dailyWithdrawalLimit: string;

  @Column({ type: 'enum', enum: PlayerWalletStatus })
  status: PlayerWalletStatus;
}
```

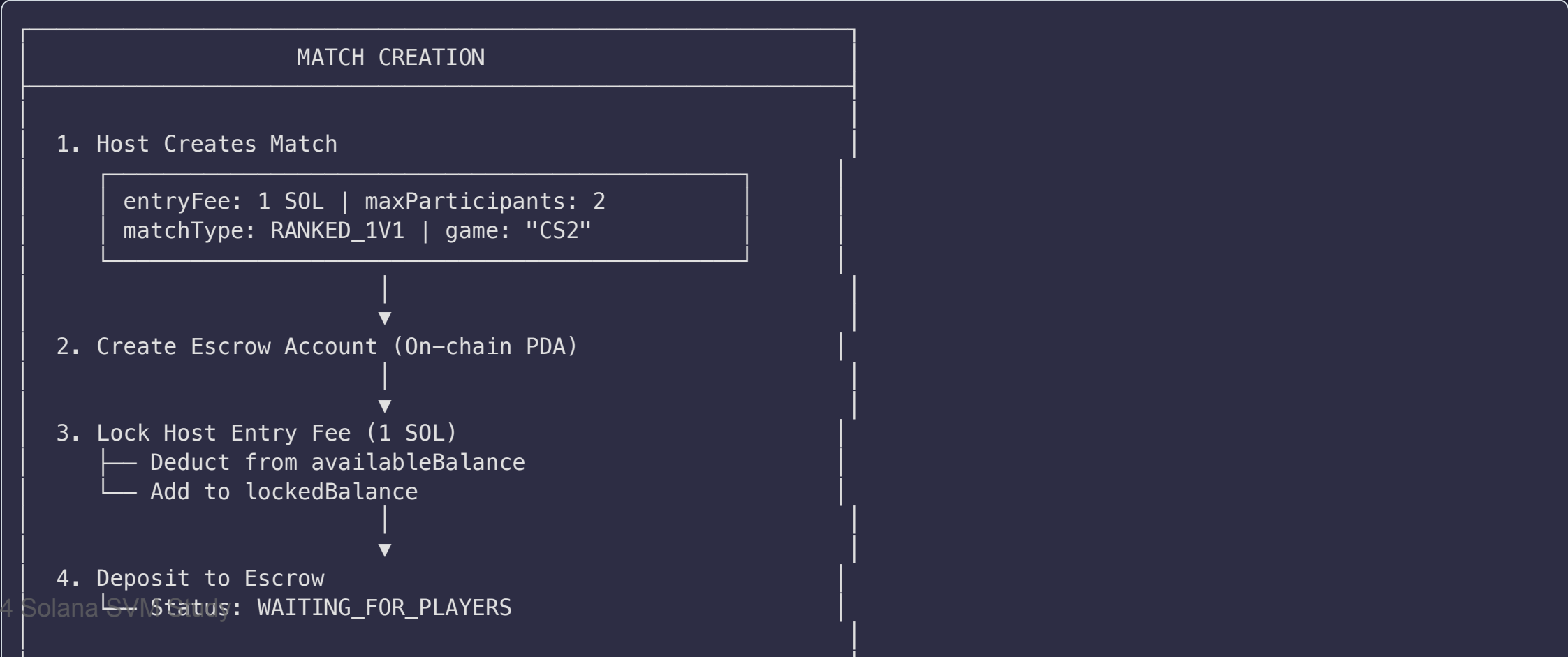
Wallet Operations Flow





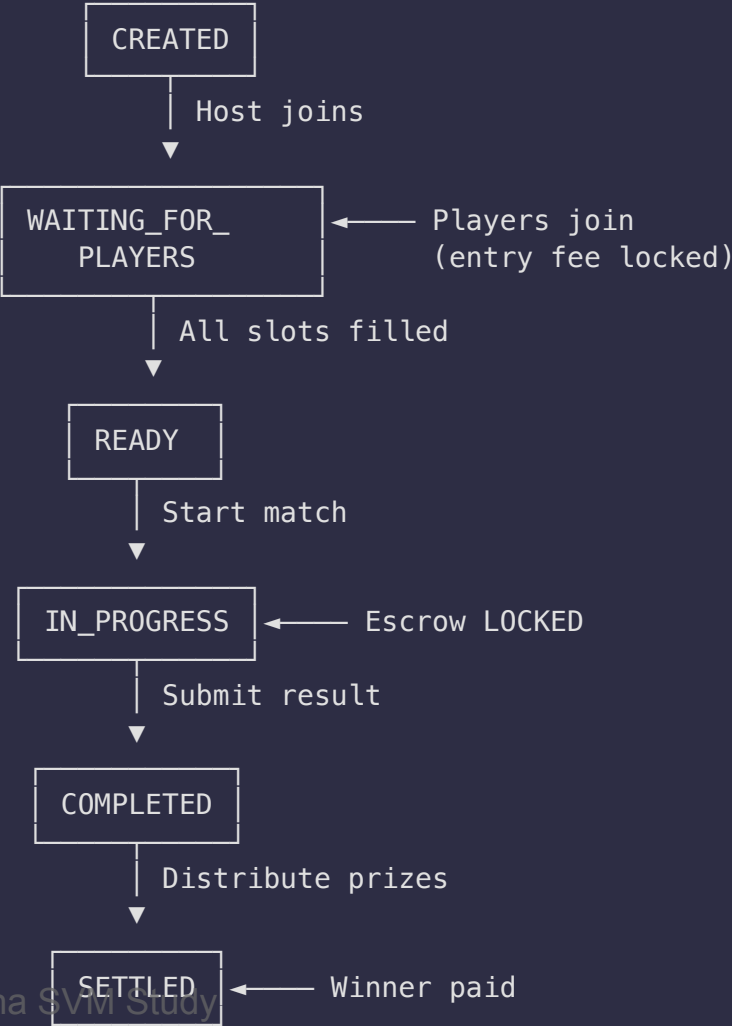
Matchmaking System

Entry Fee & Escrow Flow





Match Lifecycle



Prize Distribution Logic

Winner Takes Pool (minus platform fee)

```
async distributeMatchPrizes(matchId: string) {
  const match = await this.matchRepository.findOne({
    where: { id: matchId },
    relations: ['escrow', 'participants'],
  });

  // Calculate prize pool
  const totalPool = BigInt(match.entryFee) *
    BigInt(match.participants.length);

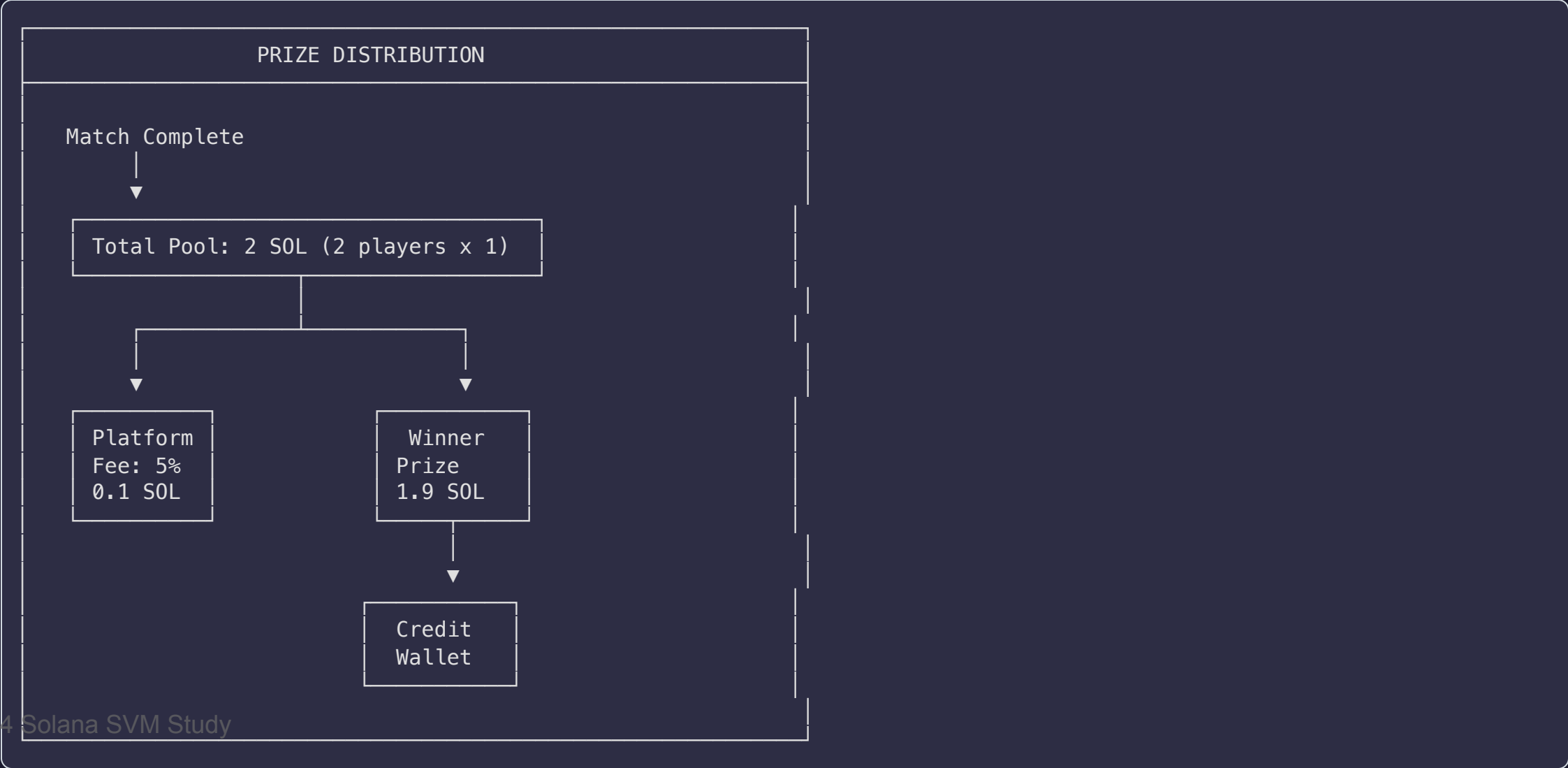
  // Deduct platform fee (e.g., 5%)
  const platformFee = totalPool * BigInt(match.platformFeePercentage) / 100n;
  const winnerPrize = totalPool - platformFee;

  // Credit winner's wallet
  await this.walletService.creditPrize(
    match.winnerId,
    winnerPrize.toString(),
    matchId
  );

  // Release escrow
  await this.escrowRepository.releaseEscrow(matchId);
}
```



Prize Distribution Flow





Tournament System

Bracket Generation

```
async generateBracket(tournamentId: string): Promise<Tournament> {
  const tournament = await this.tournamentRepository.findOne(/*...*/);
  const registrations = await this.registrationRepository.find({
    where: { tournamentId },
    order: { seed: 'ASC' },
  });

  // Single elimination: seeded matchups
  // 1 vs 8, 2 vs 7, 3 vs 6, 4 vs 5
  const bracket = this.createSingleEliminationBracket(
    registrations,
    tournament.maxParticipants
  );

  // Create match entities for each bracket slot
  const matches = await this.createBracketMatches(
    tournament,
    bracket
  );

  return this.tournamentRepository.save({
    ...tournament,
  });
}
```

Tournament Bracket Structure



Tournament Prize Structure

```
const prizeStructure = {
  1: 50, // 1st place: 50% of pool
  2: 30, // 2nd place: 30% of pool
  3: 20, // 3rd/4th: 20% split
};

// Example: 16 players x 10 SOL entry = 160 SOL pool
// Platform fee (5%): 8 SOL
// Distributable: 152 SOL

// Payouts:
// 1st: 76 SOL
// 2nd: 45.6 SOL
// 3rd: 30.4 SOL
```

Placement	Percentage	Prize (SOL)
 1st	50%	76.0
 2nd	30%	45.6
 3rd	20%	30.4

Escrow Account Entity

```
@Entity('escrow_accounts')
export class EscrowAccount {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ type: 'enum', enum: EscrowType })
  type: EscrowType; // MATCH | TOURNAMENT

  @Column()
  reference: string; // match_id or tournament_id

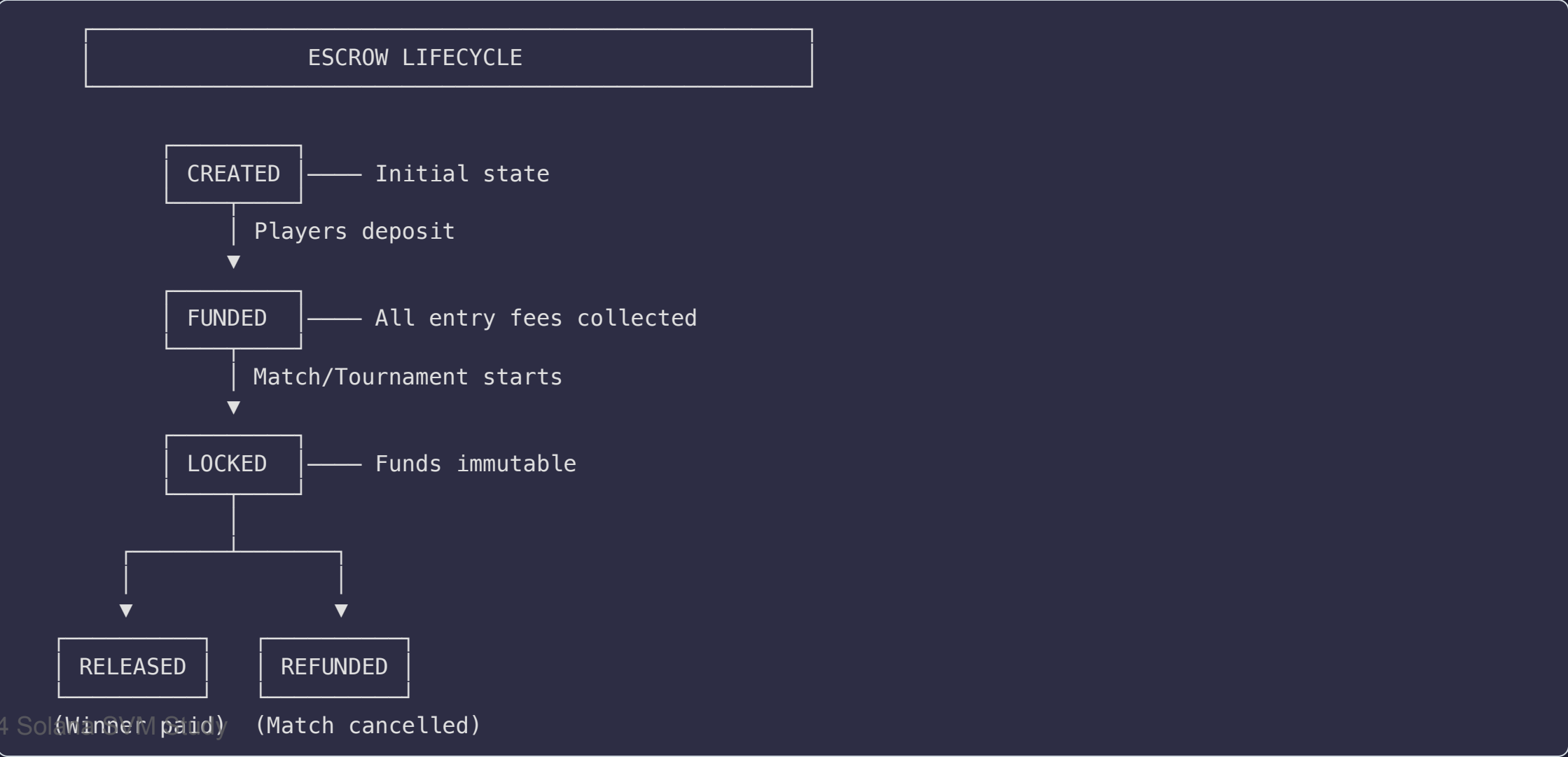
  @Column()
  escrowPda: string; // Program Derived Address

  @Column({ type: 'decimal', precision: 20, scale: 0 })
  amount: string;

  @Column({ type: 'enum', enum: EscrowStatus })
  status: EscrowStatus;
  // CREATED → FUNDED → LOCKED → RELEASED/REFUNDED
}
```



Escrow State Machine



Security Measures

Production Considerations

Wallet Security

-  MPC 2-of-3 threshold signatures
-  Daily withdrawal limits
-  Rate limiting on transactions
-  Account suspension capabilities

Escrow Security

-  On-chain PDA escrow accounts
-  Atomic state transitions
-  Immutable locked funds



Rate Limiting Implementation

```
@Entity('player_wallets')
export class PlayerWallet {
  @Column({ type: 'decimal', precision: 20, scale: 0 })
  dailyWithdrawalLimit: string;

  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  dailyWithdrawalAmount: string;

  @Column({ type: 'timestamp', nullable: true })
  dailyWithdrawalResetAt: Date;

  canWithdraw(amount: bigint): boolean {
    if (this.status !== PlayerWalletStatus.ACTIVE) return false;

    const available = BigInt(this.availableBalance);
    if (amount > available) return false;

    // Check daily limit
    const dailyUsed = BigInt(this.dailyWithdrawalAmount);
    const dailyLimit = BigInt(this.dailyWithdrawalLimit);

    return (dailyUsed + amount) <= dailyLimit;
  }
}
```



Transaction Types

```
export enum WalletTransactionType {  
  // Inflows  
  DEPOSIT = 'DEPOSIT',           // External deposit  
  PRIZE_WIN = 'PRIZE_WIN',       // Match/tournament win  
  REFUND = 'REFUND',             // Cancelled match refund  
  
  // Outflows  
  WITHDRAWAL = 'WITHDRAWAL',     // External withdrawal  
  ENTRY_FEE = 'ENTRY_FEE',       // Match/tournament entry  
  
  // Internal  
  LOCK = 'LOCK',                 // Lock for escrow  
  UNLOCK = 'UNLOCK',            // Release from escrow  
}
```

Full Audit Trail

Every transaction logged with signatures and timestamps



Complete Match Flow

COMPLETE MATCH FLOW

- 1. CREATE MATCH
 - Host deposits entry fee → Escrow created
- 2. JOIN MATCH
 - Player deposits entry fee → Added to escrow
- 3. START MATCH
 - Escrow LOCKED → Players notified
- 4. PLAY GAME
 - External game server handles gameplay
- 5. SUBMIT RESULT
 - Winner declared → Match COMPLETED
- 6. DISTRIBUTE PRIZES
 - Winner wallet credited → Escrow RELEASED
- 7. SETTLEMENT
 - All balances updated → Match SETTLED



API Endpoints

Matchmaking

Method	Endpoint	Description
POST	/api/matches	Create match
POST	/api/matches/:id/join	Join match
POST	/api/matches/:id/start	Start match
POST	/api/matches/:id/result	Submit result
GET	/api/matches/open	List open matches

Wallets

Method	Endpoint	Description
POST	/api/wallets	Create MPC wallet
GET	/api/wallets/:playerId/balance	Get balance



API Endpoints (cont.)

Tournaments

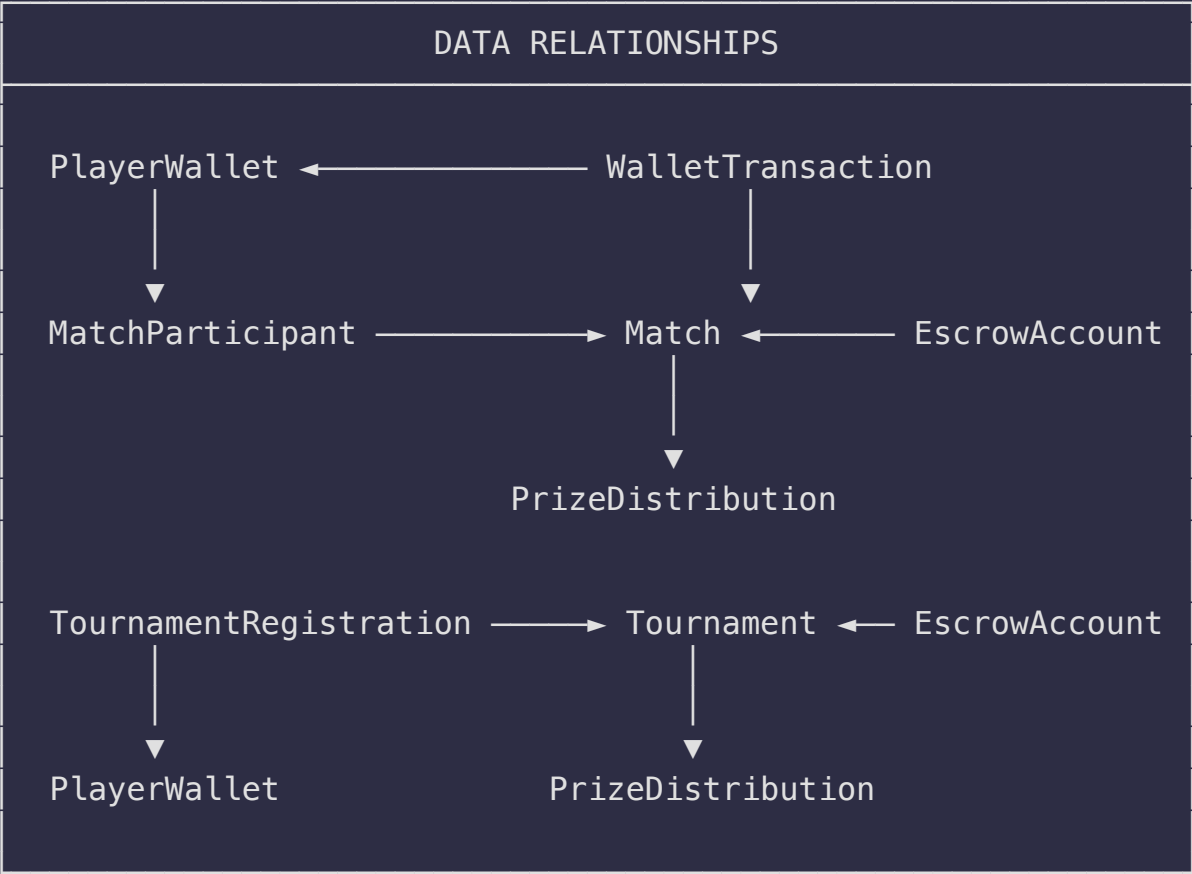
Method	Endpoint	Description
POST	/api/tournaments	Create tournament
POST	/api/tournaments/:id/register	Register player
POST	/api/tournaments/:id/bracket	Generate bracket
POST	/api/tournaments/:id/start	Start tournament
GET	/api/tournaments/:id/leaderboard	Get standings

Prizes

Method	Endpoint	Description
POST	/api/prizes/match/:id/distribute	Pay match prizes
POST	/api/prizes/tournament/:id/distribute	Pay tournament



Data Model Overview





Testing Strategy

Unit Tests

```
describe('EscrowService', () => {
  it('should create escrow for match', async () => {
    const result = await escrowService.createEscrow({
      type: EscrowType.MATCH,
      reference: 'match_123',
      tokenMint: SOL_MINT,
    });

    expect(result.status).toBe(EscrowStatus.CREATED);
    expect(result.escrowPda).toBeDefined();
  });

  it('should reject release for unlocked escrow', async () => {
    await expect(
      escrowService.releaseEscrow('escrow_unlocked')
    ).rejects.toThrow(BadRequestException);
  });
});
```



Integration Testing

```
describe('Match E2E', () => {
  it('should complete full match flow', async () => {
    // 1. Create wallets for both players
    const wallet1 = await walletService.createWallet({ playerId: 'p1' });
    const wallet2 = await walletService.createWallet({ playerId: 'p2' });

    // 2. Deposit funds
    await walletService.deposit({ playerId: 'p1', amount: '20000000000' });
    await walletService.deposit({ playerId: 'p2', amount: '20000000000' });

    // 3. Create and join match
    const match = await matchService.createMatch({
      hostPlayerId: 'p1',
      entryFee: '10000000000',
    });
    await matchService.joinMatch(match.id, 'p2');

    // 4. Start and complete
    await matchService.startMatch(match.id);
    await matchService.submitResult(match.id, { winnerId: 'p1' });

    // 5. Distribute prizes
    await prizeService.distributeMatchPrizes(match.id);

    // 6. Verify balances
    const balance1 = await walletService.getBalance('p1');
    expect(balance1.availableBalance).toBe('29000000000'); // Won
  });
});
```



Monitoring & Observability

Key Metrics to Track

```
// Custom metrics for prize distribution
@Injectable()
export class PrizeDistributionService {
  private readonly prizeDistributedCounter = new Counter({
    name: 'esports_prizes_distributed_total',
    help: 'Total prizes distributed',
    labelNames: ['type', 'status'],
  });

  private readonly prizeAmountGauge = new Gauge({
    name: 'esports_prize_amount_sol',
    help: 'Prize amount in SOL',
    labelNames: ['type'],
  });

  async distributePrize(/*...*/) {
    // ... distribution logic

    this.prizeDistributedCounter.inc({ type: 'match', status: 'success' });
    this.prizeAmountGauge.set({ type: 'match' }, prizeInSol);
  }
}
```



Key Metrics Dashboard

Metric	Description	Alert Threshold
<code>escrow_balance_total</code>	Total SOL in escrow	> 1000 SOL
<code>match_completion_rate</code>	% matches completed	< 90%
<code>prize_distribution_latency</code>	Time to distribute	> 30s
<code>mpc_signing_failures</code>	Failed MPC signs	> 0
<code>withdrawal_rate_limit_hits</code>	Rate limit triggers	> 100/hr

Alerts

- 🚨 Escrow balance mismatch
- 🚨 MPC signing failure rate > 1%
- 🚨 Prize distribution backlog
- 🚨 Suspicious withdrawal patterns

Best Practices Summary

1. Security First

- Always use MPC for player wallets
- Lock escrow before match starts
- Implement withdrawal limits

2. Atomic Operations

- Use database transactions
- Handle partial failures gracefully
- Implement idempotency keys

3. Auditability

- Log all financial transactions

Future Enhancements

Roadmap Items

1. Cross-Game Wallet

- Single wallet across multiple games
- Unified balance management

2. NFT Integration

- Tournament trophy NFTs
- Achievement badges

3. DAO Governance

- Community-driven prize pools
- Voting on platform fees



Advanced Calculation Patterns

BigInt Safety for Financial Operations

```
// CRITICAL: Always use BigInt for lamport calculations
// JavaScript Number.MAX_SAFE_INTEGER = 9,007,199,254,740,991
// 1 SOL = 1,000,000,000 lamports
// Max safe: ~9,007 SOL with regular Number

// BAD: Number overflow risk
const pool = 10000000000 * 2; // Potential precision loss

// GOOD: BigInt for all financial math
const pool = BigInt('10000000000') * BigInt(2);

// Division truncates (floor), handle remainder
const prizePerWinner = distributableAmount / BigInt(winnerCount);
const remainder = distributableAmount % BigInt(winnerCount);
// Option: Add remainder to 1st place winner
```



Atomic Transaction Pattern

Database + Blockchain Consistency

```
async distributeMatchPrizes(match: Match): Promise<PrizeDistribution> {
  const queryRunner = this.dataSource.createQueryRunner();
  await queryRunner.connect();
  await queryRunner.startTransaction();

  try {
    // 1. Create prize distribution record (PROCESSING)
    const prizeDistribution = await queryRunner.manager.save(
      PrizeDistribution, { status: 'PROCESSING', ... }
    );

    // 2. Release escrow (on-chain operation)
    await this.escrowService.releaseEscrow({ escrowId, distributions });

    // 3. Credit each winner's wallet
    for (const dist of distributions) {
      await this.playerWalletService.creditPrize(dist.playerId, dist.amount);

      // 4. Update distribution status individually
      dist.status = 'COMPLETED';
      dist.processedAt = new Date();
    }

    // 5. Commit all database changes
    prizeDistribution.status = 'COMPLETED';
    await queryRunner.manager.save(prizeDistribution);
    await queryRunner.commitTransaction();

    return prizeDistribution;
  } catch (error) {
    await queryRunner.rollbackTransaction();
    throw error;
  }
}
```

Error Recovery Strategies

Partial Distribution Handling

```
// Prize distribution supports partial success
for (const dist of distributions) {
  try {
    await this.playerWalletService.creditPrize(
      dist.playerId, dist.amount, sourceId
    );

    distEntry.status = 'completed';
    distEntry.processedAt = new Date();
    totalDistributed += BigInt(dist.amount);

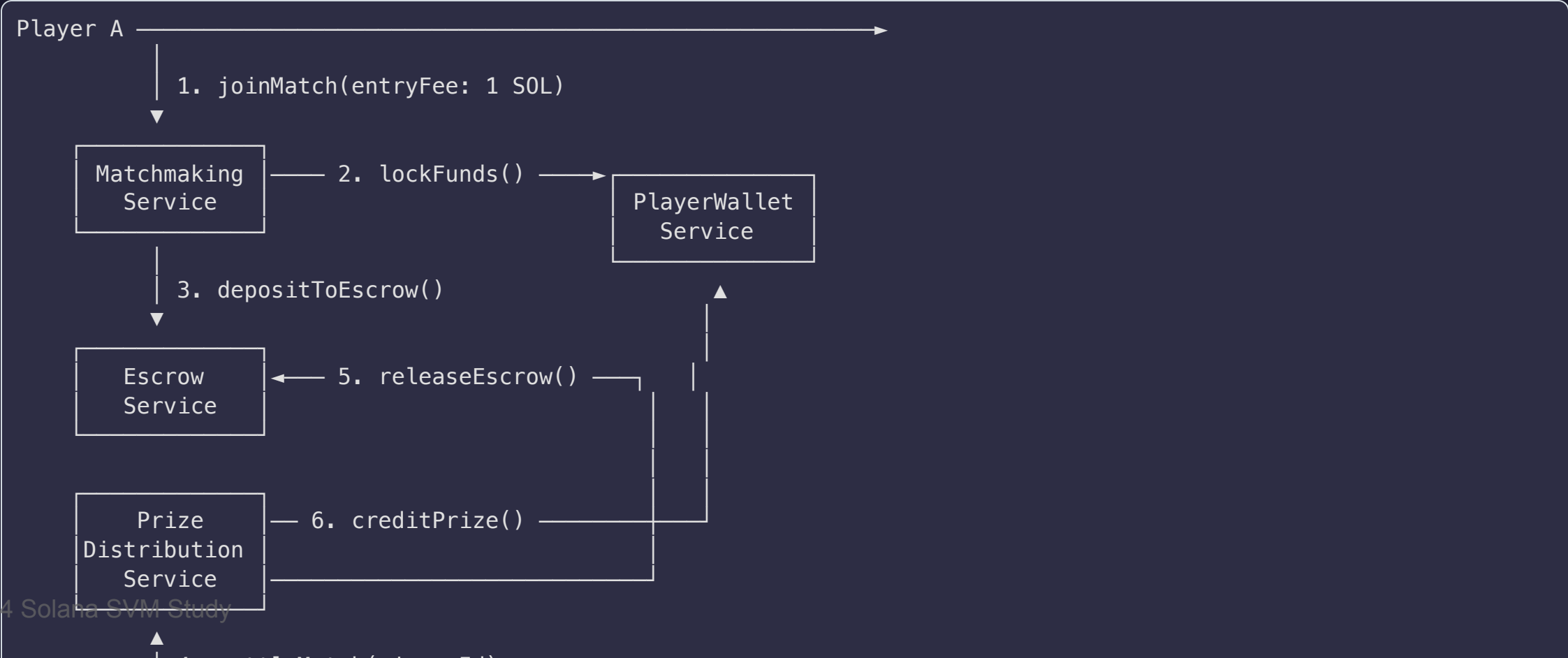
  } catch (error) {
    // Log but continue to next recipient
    this.logger.error(`Failed to credit ${dist.playerId}`, error);

    distEntry.status = 'failed';
    distEntry.failureReason = error.message;
    // Can be retried later via admin action
  }
}
```



Service Interaction Diagram

Complete Prize Distribution Flow



❖ Business Rules: Entry Fee Validation

Configurable Fee Boundaries

```
// Environment-based fee limits (lamports)
const MIN_ENTRY_FEE = BigInt(process.env.MIN_ENTRY_FEE || '1000000'); // 0.001 SOL
const MAX_ENTRY_FEE = BigInt(process.env.MAX_ENTRY_FEE || '100000000000'); // 100 SOL

// Validation rules applied at match creation
async validateEntryFee(entryFee: string, gameType: string): Promise<void> {
  const fee = BigInt(entryFee);

  // Rule 1: Minimum threshold (prevent spam)
  if (fee < MIN_ENTRY_FEE) {
    throw new BadRequestException(`Entry fee below minimum: ${MIN_ENTRY_FEE}`);
  }

  // Rule 2: Maximum cap (risk management)
  if (fee > MAX_ENTRY_FEE) {
    throw new BadRequestException(`Entry fee exceeds maximum: ${MAX_ENTRY_FEE}`);
  }

  // Rule 3: Game-specific limits
  const gameLimit = await this.getGameFeeLimit(gameType);
  if (fee > gameLimit) {
    throw new BadRequestException(`Fee exceeds game limit for ${gameType}`);
  }
}
```

Business Rules: Player Eligibility

Multi-Factor Verification

```
async verifyPlayerEligibility(playerId: string, matchId: string): Promise<void> {
  const player = await this.playerService.getPlayer(playerId);
  const wallet = await this.walletService.getWallet(playerId);
  const match = await this.matchService.getMatch(matchId);

  // Rule 1: Account status
  if (player.status !== PlayerStatus.ACTIVE) {
    throw new ForbiddenException('Account suspended or inactive');
  }

  // Rule 2: KYC verification (for high-value matches)
  if (BigInt(match.entryFee) > KYC_THRESHOLD && !player.kycVerified) {
    throw new ForbiddenException('KYC required for high-value matches');
  }

  // Rule 3: Region restrictions
  if (match.allowedRegions && !match.allowedRegions.includes(player.region)) {
    throw new ForbiddenException('Match restricted to specific regions');
  }

  // Rule 4: Skill rating bounds (prevent smurfing)
  if (match.minRating && player.skillRating < match.minRating) {
    throw new ForbiddenException(`Minimum rating required: ${match.minRating}`);
  }

  // Rule 5: Concurrent match limit
  const activeMatches = await this.matchService.getActiveMatches(playerId);
```


Business Rules: Eligibility Criteria

Player Status Requirements

Criterion	Requirement	Enforcement
Account Age	> 24 hours	Prevent bot spam
Email Verified	Required	Identity verification
Phone Verified	High-stakes only	2FA for withdrawals
KYC Level 1	Matches > 10 SOL	Regulatory compliance
KYC Level 2	Matches > 50 SOL	Enhanced due diligence
Skill Rating	Within ± 200 ELO	Fair matchmaking
Active Bans	None	Anti-cheat compliance
Pending Disputes	None	Dispute resolution first

```
© 2024 Solana SVM Study
// Eligibility flags stored on player entity
interface PlayerEligibility {
  canPlayCasual: boolean; // Basic account status
```

Business Rules: Refund Policies

Scenario-Based Refund Logic

```
enum RefundReason {  
    MATCH_CANCELLED = 'MATCH_CANCELLED',           // Full refund  
    OPPONENT_NO_SHOW = 'OPPONENT_NO_SHOW',         // Full refund + bonus  
    TECHNICAL_FAILURE = 'TECHNICAL_FAILURE',       // Full refund  
    PLAYER_DISCONNECT = 'PLAYER_DISCONNECT',        // Partial (time-based)  
    DISPUTE_RESOLVED = 'DISPUTE_RESOLVED',         // Per arbitration  
    TOURNAMENT_CANCELLED = 'TOURNAMENT_CANCELLED', // Full refund  
}  
  
async processRefund(escrowId: string, reason: RefundReason): Promise<void> {  
    const escrow = await this.escrowService.getEscrow(escrowId);  
  
    switch (reason) {  
        case RefundReason.MATCH_CANCELLED:  
            // Rule: 100% refund to all participants  
            await this.refundAllParticipants(escrow, 100);  
            break;  
  
        case RefundReason.OPPONENT_NO_SHOW:  
            // Rule: 100% refund + 10% bonus from platform pool  
            await this.refundWithBonus(escrow, waitingPlayer, 10);  
            break;  
  
        case RefundReason.PLAYER_DISCONNECT:  
            // Rule: Partial refund based on match progress  
            const refundPercent = this.calculateDisconnectRefund(escrow);
```



Business Rules: Refund Matrix

Refund Percentage by Scenario

Scenario	Refund %	Waiting Time	Platform Action
Match cancelled (< 1 min)	100%	Immediate	Auto-refund
Match cancelled (> 1 min)	100%	5 min	Admin approval
Opponent no-show	100% + bonus	10 min wait	Auto after timeout
Technical failure	100%	24 hr review	Manual verification
Disconnect (< 25% match)	75%	48 hr	Dispute period
Disconnect (25-50% match)	50%	48 hr	Dispute period
Disconnect (> 50% match)	0%	N/A	Forfeit
Rage quit	0%	N/A	Forfeit + penalty
Tournament cancelled	100%	24 hr	Batch refund

Business Rules: Match Cancellation

Cancellation Conditions & Consequences

```
enum CancellationReason {
  HOST_CANCELLED = 'HOST_CANCELLED',
  INSUFFICIENT_PLAYERS = 'INSUFFICIENT_PLAYERS',
  MATCH_TIMEOUT = 'MATCH_TIMEOUT',
  ADMIN_CANCELLED = 'ADMIN_CANCELLED',
  SERVER_UNAVAILABLE = 'SERVER_UNAVAILABLE',
}

async cancelMatch(matchId: string, reason: CancellationReason): Promise<void> {
  const match = await this.matchService.getMatch(matchId);

  // Rule 1: Only cancel if not IN_PROGRESS or later
  if (!match.isCancellable()) {
    throw new BadRequestException('Match cannot be cancelled after start');
  }

  // Rule 2: Host penalty for frequent cancellations
  if (reason === CancellationReason.HOST_CANCELLED) {
    const recentCancels = await this.getCancellationCount(match.hostId, 24);
    if (recentCancels >= 3) {
      await this.applyHostPenalty(match.hostId); // 24hr match creation ban
    }
  }

  // Rule 3: Auto-refund all participants
  await this.escrowService.refundEscrow({
    escrowId: match.escrowId,
    reason,
    distributions: match.participants.map(p => ({
      playerId: p.playerId,
      amount: match.entryFee,
    })),
  });
}
```



Business Rules: Cancellation Penalties

Host Accountability System

Cancellations (24hr)	Penalty	Duration
1-2	Warning	N/A
3-4	24hr match creation ban	24 hours
5+	7-day ban + rating penalty	7 days
Repeat offender	Account review	Indefinite

```
// Penalty escalation logic
async applyHostPenalty(playerId: string): Promise<void> {
  const cancelHistory = await this.getCancellationHistory(playerId);
  const recent = cancelHistory.filter(c => c.age < 7 * 24 * 60 * 60 * 1000);

  if (recent.length >= 10) {
    // Severe: Account review required
    await this.flagForReview(playerId, 'EXCESSIVE_CANCELLATIONS');
    await this.suspendMatchCreation(playerId, 'INDEFINITE');
  } else if (recent.length >= 5) {
    // Major: 7-day ban + rating penalty
```

Business Rules: Dispute Resolution

Multi-Tier Dispute System

```
enum DisputeStatus {
    OPENED = 'OPENED',
    EVIDENCE_COLLECTION = 'EVIDENCE_COLLECTION',
    UNDER_REVIEW = 'UNDER_REVIEW',
    ARBITRATION = 'ARBITRATION',
    RESOLVED = 'RESOLVED',
    APPEALED = 'APPEALED',
}

interface Dispute {
    disputeId: string;
    matchId: string;
    claimantId: string;    // Player filing dispute
    respondentId: string;  // Other player
    category: DisputeCategory;
    evidence: Evidence[];
    status: DisputeStatus;
    tier: DisputeTier;      // AUTO → SUPPORT → ARBITRATION
    resolution?: DisputeResolution;
}

enum DisputeCategory {
    CHEATING = 'CHEATING',
    DISCONNECT = 'DISCONNECT',
    WRONG_RESULT = 'WRONG_RESULT',
}
```

Business Rules: Dispute Tiers

Escalation Ladder



Business Rules: Dispute Evidence

Required Evidence Types

Business Rules: Timeout Handling

Match Lifecycle Timeouts

```
// Configurable timeout values
const TIMEOUTS = {
  MATCH_FILL: 30 * 60 * 1000,      // 30 min to fill match
  PLAYER_READY: 5 * 60 * 1000,     // 5 min ready check
  MATCH_START: 10 * 60 * 1000,     // 10 min to connect to game
  RESULT_SUBMIT: 30 * 60 * 1000,    // 30 min after match end
  DISPUTE_WINDOW: 24 * 60 * 60 * 1000, // 24 hr dispute window
  PRIZE_CLAIM: 30 * 24 * 60 * 60 * 1000, // 30 days to claim
};

@cron('*/*5 * * * *') // Every 5 minutes
async handleMatchTimeouts(): Promise<void> {
  // Rule 1: Cancel unfilled matches
  const unfilledMatches = await this.matchRepository.find({
    where: {
      status: MatchStatus.WAITING_FOR_PLAYERS,
      createdAt: LessThan(new Date(Date.now() - TIMEOUTS.MATCH_FILL)),
    },
  });
};
```

```
function cancelMatch of unfilledMatches) {
  await this.cancelMatch(match.id, CancellationReason.MATCH_TIMEOUT);
}
```



Business Rules: Timeout Actions

Automatic Timeout Handlers

Timeout	Trigger	Action	Notification
Match Fill (30m)	No players join	Cancel + refund	Host notified
Ready Check (5m)	Player not ready	Kick + refund	Warning sent
Game Connect (10m)	No connection	Auto-forfeit	Opponent wins
Result Submit (30m)	No result	Admin escalation	Both notified
Dispute Window (24h)	No dispute	Distribute prizes	Final settlement
Prize Claim (30d)	Unclaimed prize	Move to reserve	Last chance email

```
// Ready check timeout handling
async handleReadyTimeout(matchId: string): Promise<void> {
  const match = await this.matchService.getMatch(matchId);
  const notReady = match.participants.filter(p => !p.isReady);
  for (const player of notReady) {
    // Remove player and refund
  }
}
```

Business Rules: Tournament Registration

Registration Constraints

```
async registerForTournament(  
  tournamentId: string,  
  playerId: string  
) : Promise<TournamentRegistration> {  
  const tournament = await this.getTournament(tournamentId);  
  const player = await this.playerService.getPlayer(playerId);  
  
  // Rule 1: Registration window  
  if (new Date() > tournament.registrationDeadline) {  
    throw new BadRequestException('Registration closed');  
  }  
  
  // Rule 2: Capacity check  
  const registrations = await this.getRegistrationCount(tournamentId);  
  if (registrations >= tournament.maxParticipants) {  
    // Add to waitlist if enabled  
    if (tournament.waitlistEnabled) {  
      return this.addToWaitlist(tournamentId, playerId);  
    }  
    throw new BadRequestException('Tournament full');  
  }  
  
  // Rule 3: Eligibility (skill bounds, KYC, etc.)  
  await this.verifyTournamentEligibility(player, tournament);  
  
  // Rule 4: No duplicate registration  
  const existing = await this.findRegistration(tournamentId, playerId);  
  if (existing) {  
    throw new ConflictException('Already registered');  
  }  
}
```

Business Rules: Tournament Constraints

Registration Requirements

Constraint	Rule	Exception
Registration Window	Opens 7d before, closes 1hr before	Admin override
Min Players	Tournament starts only if min reached	Cancel + refund
Max Players	First-come-first-served	Waitlist fills drops
Entry Fee Lock	Locked at registration	Refund if withdrawn early
Withdrawal Deadline	24hr before start	100% refund
Late Withdrawal	< 24hr before start	50% refund
No-Show	Not present at start	0% refund + ban

```
// Withdrawal refund calculation
calculateWithdrawalRefund(tournament: Tournament, requestTime: Date): number {
  const hoursUntilStart = (tournament.startTime - requestTime) / (60 * 60 * 1000);

  if (hoursUntilStart > 24) return 100; // Full refund
```

Business Rules: Prize Pool Guarantees

Guaranteed Prize Pools (GPP)

```
interface GuaranteedPrizePool {
  guaranteedAmount: string; // Minimum prize pool
  actualPool: string; // Entry fees collected
  overlayAmount: string; // Platform covers shortfall
  isGuaranteed: boolean;
}

async calculatePrizePool(tournament: Tournament): Promise<GuaranteedPrizePool> {
  const registrations = await this.getRegistrationCount(tournament.id);
  const actualPool = BigInt(tournament.entryFee) * BigInt(registrations);
  const guaranteedAmount = BigInt(tournament.guaranteedPrizePool || '0');

  // Platform covers shortfall (overlay)
  const overlayAmount = guaranteedAmount > actualPool
    ? guaranteedAmount - actualPool
    : BigInt(0);

  return {
    guaranteedAmount: guaranteedAmount.toString(),
    actualPool: actualPool.toString(),
    overlayAmount: overlayAmount.toString(),
    isGuaranteed: tournament.guaranteedPrizePool != null,
  };
}
```

Business Rules: GPP Economics

Guaranteed Pool Scenarios

Scenario	Entry Fee	Expected	Actual	Overlay	Result
Under-filled	10 SOL	100 players	80	200 SOL	GPP met
Exactly met	10 SOL	100 players	100	0 SOL	GPP met
Over-filled	10 SOL	100 players	120	0 SOL	+200 SOL

```
// Prize pool calculation with GPP
async finalizePrizePool(tournament: Tournament): Promise<void> {
  const { actualPool, guaranteedAmount, overlayAmount } =
    await this.calculatePrizePool(tournament);

  // If overlay required, transfer from platform reserve
  if (BigInt(overlayAmount) > 0) {
    await this.transferFromPlatformReserve(
      tournament.escrowId,
      overlayAmount
    );

    // Log overlay for accounting
    await this.logOverlay({
```

Business Rules: Anti-Collusion

Collusion Detection Mechanisms

```
interface CollusionIndicator {
  type: CollusionType;
  severity: 'LOW' | 'MEDIUM' | 'HIGH';
  confidence: number; // 0-100
  evidence: string[];
}

enum CollusionType {
  CHIP_DUMPING = 'CHIP_DUMPING', // Intentionally losing
  WIN_TRADING = 'WIN_TRADING', // Alternating wins
  MULTI_ACCOUNTING = 'MULTI_ACCOUNTING', // Same person, multiple accounts
  SOFT_PLAYING = 'SOFT_PLAYING', // Not competing when matched
  GHOSTING = 'GHOSTING', // External assistance
}

async detectCollusion(matchId: string): Promise<CollusionIndicator[]> {
  const indicators: CollusionIndicator[] = [];
  const match = await this.matchService.getMatch(matchId);

  // Check 1: Win trading (same players, alternating wins)
  const history = await this.getMatchHistory(match.participants);
  if (this.detectWinTrading(history)) {
    indicators.push({
      type: CollusionType.WIN_TRADING,
      severity: 'HIGH',
      confidence: 85,
      evidence: ['10 matches, 5-5 split, suspicious timing'],
    });
  }
}
```

Business Rules: Collusion Prevention

Detection Rules & Actions

Indicator	Detection Method	Action	Appeal
Same IP	Network fingerprint	Block match	Proof of separate identity
Device match	Hardware ID	Account link warning	Explain shared device
Win trading	Win/loss pattern	Flag for review	Match history audit
Chip dumping	Game log analysis	Suspend + investigate	Replay review
Soft playing	Performance metrics	Warning	Prove competitive play

```
// Pre-match collusion check
async validateMatchParticipants(matchId: string): Promise<void> {
  const match = await this.matchService.getMatch(matchId);
  const participants = match.participants;

  for (let i = 0; i < participants.length; i++) {
    for (let j = i + 1; j < participants.length; j++) {
      const p1 = participants[i];
      const p2 = participants[j];
```


Business Rules: Abandonment Handling

Match Abandonment Rules

```
enum AbandonmentType {
  DISCONNECT = 'DISCONNECT',    // Network issues
  QUIT = 'QUIT',                 // Voluntary exit
  AFK = 'AFK',                   // Away from keyboard
  CRASH = 'CRASH',               // Game/system crash
}

async handleAbandonment(
  matchId: string,
  playerId: string,
  type: AbandonmentType
): Promise<void> {
  const match = await this.matchService.getMatch(matchId);

  // Rule 1: Grace period for reconnection
  if (type === AbandonmentType.DISCONNECT || type === AbandonmentType.CRASH) {
    const gracePeriod = match.reconnectGracePeriod || 3 * 60 * 1000; // 3 min

    await this.startReconnectTimer(matchId, playerId, gracePeriod);
    return; // Wait for reconnect before forfeit
  }

  // Rule 2: Immediate forfeit for quit/AFK
  if (type === AbandonmentType.QUIT || type === AbandonmentType.AFK) {
    await this.forfeitPlayer(matchId, playerId);
    await this.applyAbandonmentPenalty(playerId, type);
  }
}
```

Business Rules: Abandonment Penalties

Progressive Penalty System

Abandonments (7d)	Type	Penalty	Cooldown
1st	Disconnect	Warning	None
1st	Quit	-10 rating	None
2nd	Any	-25 rating	1hr queue ban
3rd	Any	-50 rating	6hr queue ban
4th	Any	-100 rating	24hr queue ban
5+	Any	Account review	7d ban

```
async applyAbandonmentPenalty(  
  playerId: string,  
  type: AbandonmentType  
): Promise<void> {  
  const recentAbandons = await this.getAbandonments(playerId, 7 * 24 * 60 * 60);  
  const count = recentAbandons.length + 1;
```

Business Rules: Platform Fee Structure

Dynamic Fee Configuration

```
interface FeeStructure {
  baseFeePercent: number; // Default platform fee
  volumeDiscounts: VolumeDiscount[];
  promotionalRates: PromotionalRate[];
  gameSpecificFees: Map<string, number>;
}

// Volume-based discounts for high-value tournaments
const VOLUME_DISCOUNTS: VolumeDiscount[] = [
  { minPool: '0', maxPool: '10000000000', feePercent: 5.0 }, // < 100 SOL: 5%
  { minPool: '10000000000', maxPool: '1000000000000', feePercent: 4.0 }, // 100-1000: 4%
  { minPool: '1000000000000', maxPool: null, feePercent: 3.0 }, // > 1000 SOL: 3%
];

async calculatePlatformFee(
  prizePool: bigint,
  gameType: string,
  eventType: string
): Promise<bigint> {
  // Base fee from volume tier
  let feePercent = this.getVolumeBasedFee(prizePool);

  // Game-specific adjustment
  const gameAdjustment = this.gameSpecificFees.get(gameType) || 0;
  feePercent += gameAdjustment;

  // Promotional override (e.g., launch discount)
  const promo = await this.getActivePromotion(eventType);
  if (promo) {
    feePercent = Math.max(feePercent - promo.discountPercent, 1.0);
  }
}
```

Business Rules: Fee Examples

Platform Fee Calculation Table

Pool Size	Base Fee	Volume Discount	Promo	Final Fee	Platform Revenue
2 SOL	5.0%	None	None	5.0%	0.10 SOL
50 SOL	5.0%	None	-1% launch	4.0%	2.00 SOL
200 SOL	4.0%	-1% (>100)	None	4.0%	8.00 SOL
500 SOL	4.0%	-1% (>100)	-1% launch	3.0%	15.00 SOL
2000 SOL	3.0%	-2% (>1000)	None	3.0%	60.00 SOL

```
// Fee breakdown for transparency
interface FeeBreakdown {
  totalPool: string;
  baseFeePercent: number;
  volumeDiscount: number;
  promotionalDiscount: number;
  finalFeePercent: number;
  platformFee: string;
  distributableAmount: string;
```

Resources

Documentation

- [Solana Web3.js Docs](#)
- [SPL Token Program](#)
- [MPC Cryptography](#)

Code References

- [/src/modules/esports/](#) - Full implementation
- [/docs/diagrams/16-esports-matchmaking.md](#) - Architecture docs
- [/src/modules/mpc/](#) - MPC wallet integration

Tools

- [Anchor Framework](#) for Solana programs

? Questions?

Key Takeaways

1. **MPC wallets** provide security without UX friction
2. **Escrow accounts** ensure trustless prize distribution
3. **Atomic state machines** prevent fund loss
4. **Platform co-signing** enables fraud prevention
5. **Full audit trails** support dispute resolution

 **Thank You!**

Contact & Resources

-  Project Repository: github.com/psavelis/solana-svm-study
-  Documentation: </docs/diagrams/16-esports-matchmaking.md>
-  Tests: /src/modules/esports/__tests__/

 **Let's Build!**

Production-Ready Esports on Solana